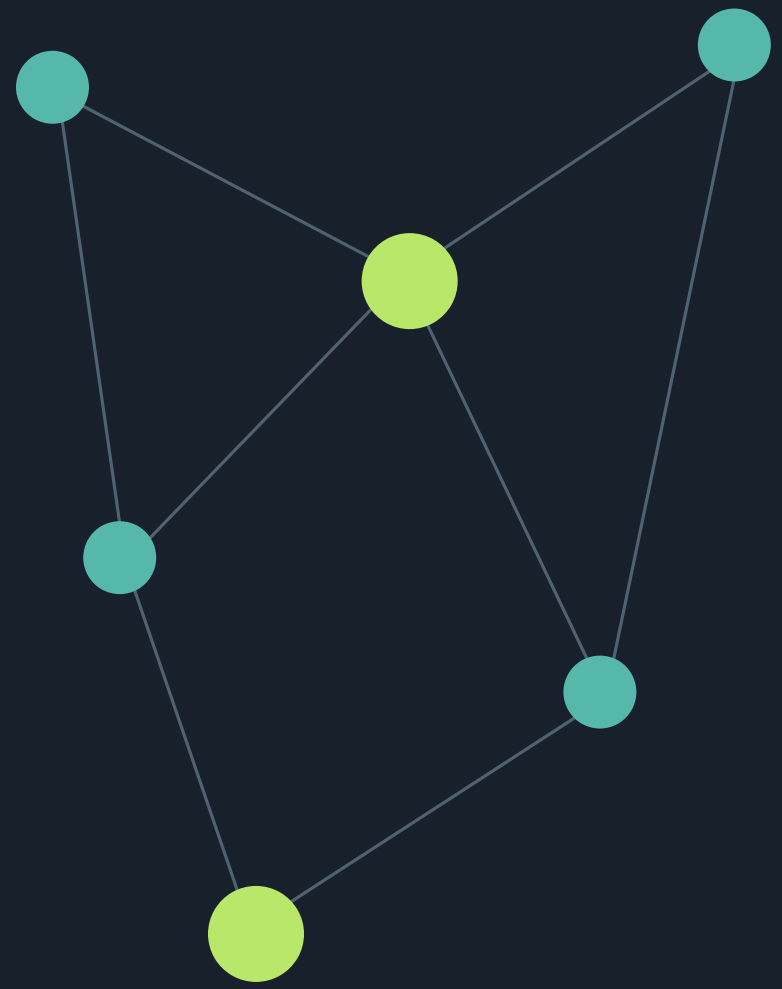
WARDEN

Verifiable delivery for Kubernetes

A delivery platform that connects build provenance, policy checks, progressive rollout and recovery.

CCD MINI PROJECT • BATCH C2

Abhyuday Gupta · Aditya Chetan Thakur · Sankalp Arya · Shaurya Jain



From source commit
to healthy workload

Project results and remaining verification gates · 27 October 2026

WHY WARDEN

Software delivery has three connected trust gaps

A signature is useful only when the build, admission decision and runtime behavior are connected.

Origin

Can the running image be traced to a reviewed source change and a known build workflow?

Admission

Will the cluster reject an image that is unsigned, altered or produced by an untrusted builder?

Runtime

Can a faulty release be detected and rolled back before it affects most requests?



Evidence should follow the artifact all the way to the workload.

SYNOPSIS OUTCOMES

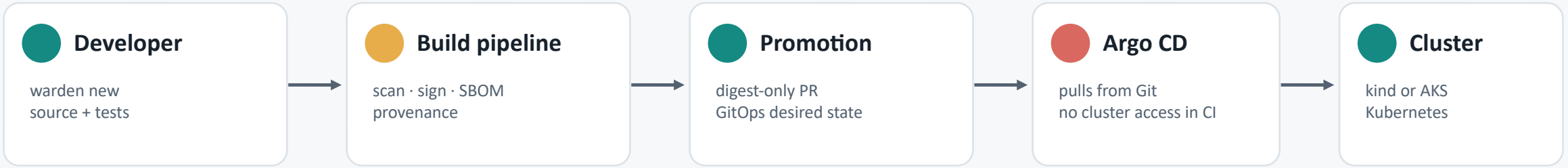
Four project outcomes were measured

The scorecard separates verified behavior from the remaining managed-database and enforcement gates.

01	**Create a service**	Five complete kind runs; median 8.6 min; zero manual steps.	MEASURED
02	**Reject untrusted images**	15/15 controlled attack cases refused on kind and AKS; live image policy remains Audit.	CONTROLLED TEST
03	**Undo a faulty release**	10/10 historical faults and 4/4 app-only injected faults aborted automatically.	MEASURED
04	**Provision a database**	CloudNativePG measured on kind and AKS; Azure Flexible Server code awaits live verification.	PARTIAL

One path connects source, policy and runtime

The sample services stay small so the delivery platform and its controls remain visible.



Admission

VAP checks immutable image references. Kyverno checks supply-chain evidence and pod hygiene.

Progressive delivery

Argo Rollouts shifts traffic and evaluates service metrics before promotion.

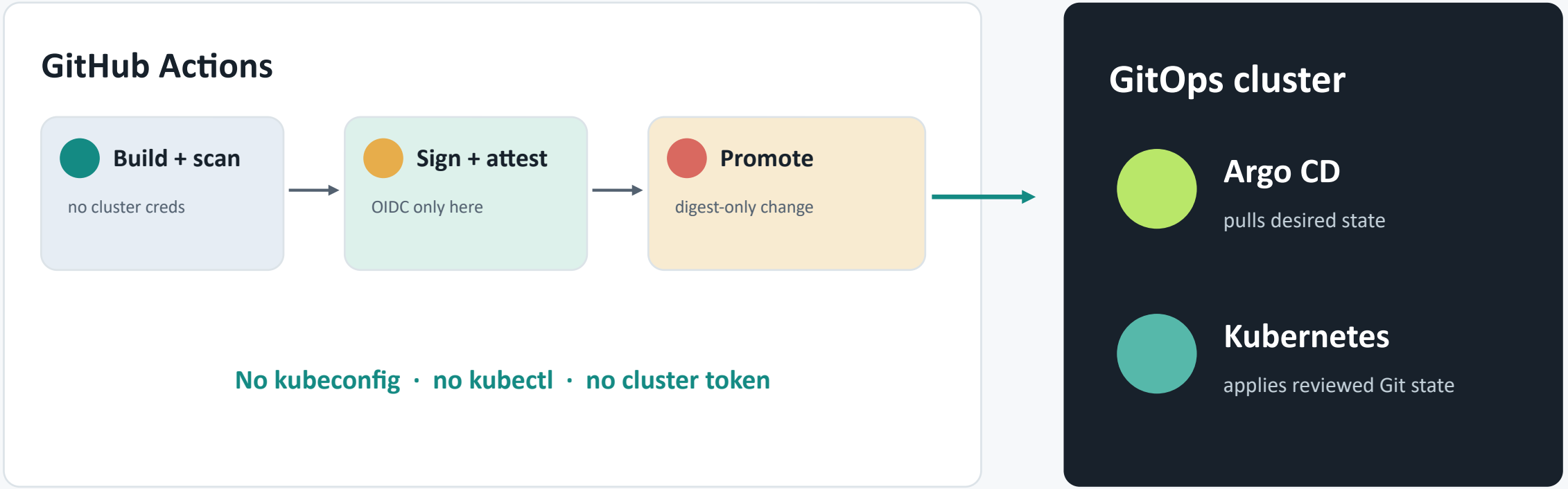
Operations

Crossplane provides a shared Database API; Prometheus and GitHub Deployments provide evidence.

TRUST BOUNDARY

CI can propose state; it cannot reach the cluster

Every image is promoted by digest through Git, then pulled by Argo CD.



Only a Git change crosses the boundary

POLICY CONTROLS

Admission is layered; cryptographic checks remain observable

Fast structural rules are enforced. Image verification stays in Audit until reliability gates pass.

01 Digest and registry

VAP: immutable digests and approved registries

DENY

02 Supply-chain proof

Kyverno: signatures, provenance and SBOM

AUDIT

03 Pod hygiene

Kyverno: workload security and runtime defaults

DENY

Observed cost: 14–15 s per warm Pod · webhook timeout: 30 s · measured worst case leaves only 10–15 s margin

A service environment appears from one command

The CLI creates the service repository and takes it through build, promotion and a healthy rollout.

5 / 5

full kind runs

all reached a healthy deployment

8.6 min

median time

range: 7.6–9.6 minutes

0

manual steps

from command to healthy service

warden new



source → rollout

Still open: Azure E1 runs and a hand-timed baseline for comparison.

The controlled attack suite rejected every case

Each adversarial image was denied for its expected reason in tests on kind and AKS.

A01 A02 A03 A04 A05 A06 A07

A08 A09 A10 A11 A12 A13 A14

15 / 15

test cases passed

including one legitimate control

Controlled Deny tests are not live enforcement.

The live image policy remains in Audit while Kyverno reliability gates are unresolved.

Faulty releases stop early; healthy releases continue

Real signed builds moved through GitOps and Argo Rollouts; the app-only batch used no owner merge.

4 / 4
app-only faults aborted
all on the success-rate check

147.6 s
median abort time
range: 145.4–149.6 s

12 / 12
healthy canaries promoted
app-only batch

0.39%
median error share
injected-fault request share



Abort → fixed release promoted: median 756.6 s

One Database contract spans local and managed backends

The Crossplane v2 API remains the same; the selected implementation changes by cluster.

MEASURED

CloudNativePG

kind: 804 s cold · 21 s warm
AKS: 138 s

Six connection keys; `psql` connected; wrong password refused.

IMPLEMENTED · LIVE TEST PENDING

Azure Flexible Server

Provider + managed identity + narrow role + one-IP firewall are prepared on review branches.

Still required: service-pod connection, readiness, delete and full teardown.

Same developer request → backend chosen by the cluster overlay

The app-only release batch produced a complete delivery dataset

Twenty scripted deployments across four services; the batch was designed to include four injected faults.

20

deployments

five per service

12.5 min

median lead time

commit to deployment record

12.6 min

median recovery

failure to fixed release

20%

planned failure rate

4 injected faults of 20

Median lead by service



Read the sample carefully

Short scripted window; synthetic changes; no human review in lead time. Frequency is not normal team cadence.

E6 · REPRODUCIBILITY

The environment rebuilds from Git on local and Azure

Each successful round returns GitOps applications to healthy and the four sample services to HTTP 200.

LOCAL · KIND

12 / 13

successful rebuilds

Median 14.8 min · p90 24.2 min

AZURE · AKS

5 / 5

successful rebuilds from empty

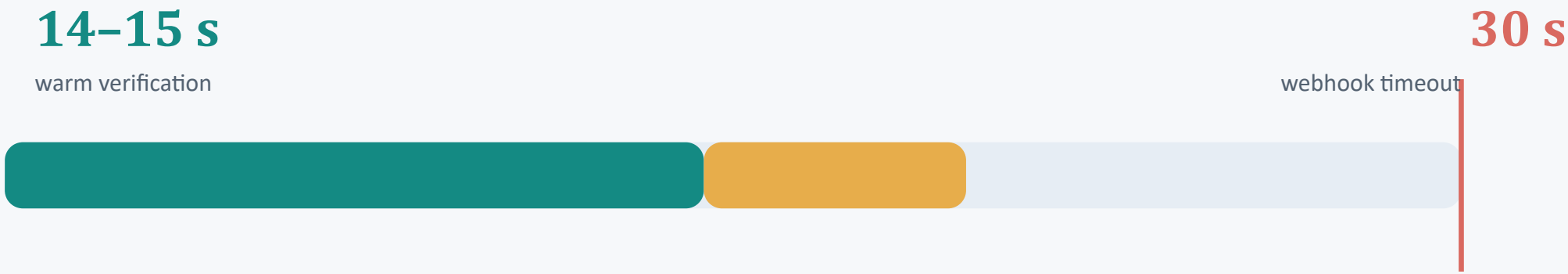
Median 22 min · zero manual steps

4 FAILED AZURE ATTEMPTS RETAINED AND EXPLAINED

A rebuild ledger records both successes and failures.

Verification latency explains why Deny is staged

The webhook budget is 30 seconds, so the warm-path margin is real but not comfortable.



Kind and AKS produced similar warm measurements.

Next gate: retry transient fetches, then repeat legitimate-pod and attack suites before changing policy mode.

ENGINEERING LESSONS

The measurements exposed defects before submission

The platform improved through observed failures, not only happy-path demonstrations.

Traffic ownership

Argo CD reset a Rollouts-managed Service selector and changed canary traffic share. The sync rules were corrected and forced-sync verified.

Policy exemptions

Audit reports hid that legitimate database Pods would be blocked in Deny. The test process now checks real workload classes before enforcement.

Deployment counting

Argo CD emitted several deployment records for one revision. The CLI now deduplicates promotion commits and uses final deployment status.

Failure evidence changed both the implementation and the measurement method.

SUBMISSION GATES · 27 OCT 2026

A short list remains before the project is fully closed

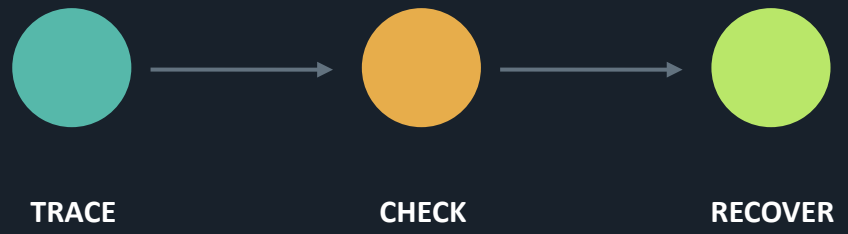
These are explicit verification and approval tasks, not features being presented as complete.

1	**Azure database**	Review the two PRs; provision a small Flexible Server; connect from a service Pod; delete it and tear down AKS.
2	**Image enforcement**	Keep Audit on the pinned Kyverno release; Deny requires five clean releases per service, attack coverage and timeout margin.
3	**Evaluation + release**	Azure E1 and hand-timed baseline are open; tags require reviewed main branches and final documentation.
4	**Presentation**	Record the four scenes, update this slide on presentation day and obtain supervisor review. The fresh local rebuild rehearsal passed.

NO KYVERNO VERSION CHANGE

A safer path from source to running software

Warden joins provenance, policy, GitOps and progressive delivery in one measured platform.



What is proven — Local and AKS rebuilds · 15 controlled attacks · canary rollback · app-only delivery metrics

What is still open — Live managed Azure database verification · reliable image-verification Deny gate

Selected foundations

Research and technical articles that informed the platform design.

- [1] Verma, A. et al. “Large-scale cluster management at Google with Borg.” EuroSys, 2015.
- [2] Torres-Arias, S. et al. “in-toto: Providing farm-to-table guarantees for bits and bytes.” USENIX Security, 2019.
- [3] SLSA. “Provenance: Build track.” Supply-chain Levels for Software Artifacts specification.
- [4] OpenGitOps. “OpenGitOps Principles.” Cloud Native Computing Foundation.
- [5] Beyer, B. et al. “Canarying Releases.” Site Reliability Engineering, Google.
- [6] Forsgren, N., Humble, J., Kim, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution, 2018.

Experiment results are documented separately in the project report and evaluation evidence.